# MediCore · Plan de pruebas

Plan formal del Proyecto 2 (curso 048, UMG). Sigue las 20 secciones de la sección 7 del documento del proyecto. Los números de la sección 18 salen de reportes guardados en `reports/`; lo que no se ejecutó aparece como pendiente.

## 1. Introducción y objetivo

MediCore centraliza pacientes, agendas, consultas, recetas, resultados de laboratorio y notificaciones de una institución de salud. Incluye una web para el personal, una app Android para el paciente y una API.

El objetivo del plan es comprobar, con evidencia reproducible, cuatro cosas:

1. Las reglas clínicas y de agenda funcionan por cada rol.
2. Ningún usuario ve o cambia información que no le corresponde.
3. Con alta demanda no hay doble reserva, y una falla del broker no pierde datos clínicos.
4. La solución se puede reconstruir desde el repositorio y el pipeline detiene una entrega con fallas.

## 2. Alcance y fuera de alcance

**Dentro del alcance**

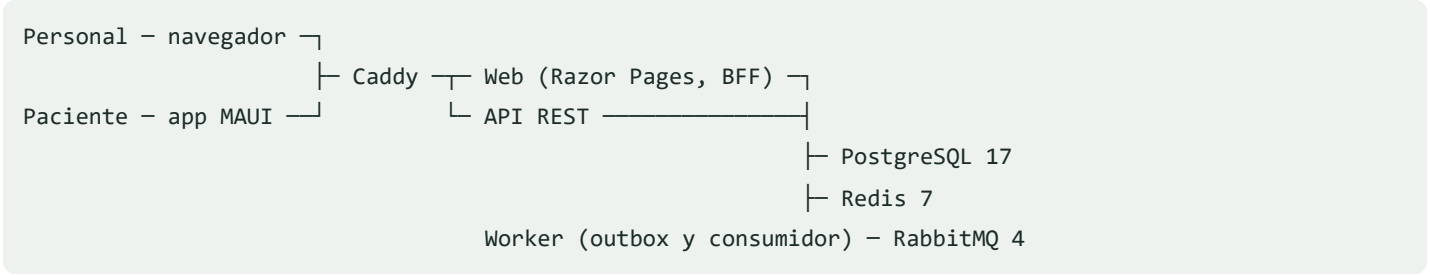
- Módulos: Identidad, Pacientes, Médicos, Agenda, Consultas, Recetas, Laboratorio, Notificaciones y Auditoría.
- Web del personal (WEB-01 a WEB-18) y app del paciente (MOB-01 a MOB-12).
- API de referencia y sus rutas complementarias, con 38 rutas en `documentacion/openapi-v1.json`.
- Redis: disponibilidad, catálogos, sesiones, TTL, invalidación y caída del servicio.
- RabbitMQ: outbox, consumidor, reintentos, DLQ e idempotencia.
- Pipeline de Jenkins, contenedores, Terraform y Ansible.

**Fuera del alcance**

- Portal web del paciente y módulos no listados en el documento.
- Microservicios.
- Integración con sistemas hospitalarios externos.
- Pagos y facturación.
- Aprovisionar Azure de pago: la decisión de pago sigue abierta. El código se valida, pero no se aplica.

## 3. Arquitectura de la solución

Monolito modular en .NET 10 con un solo modelo de datos.



- **Datos:** PostgreSQL garantiza las reglas críticas. Dos restricciones `EXCLUDE` impiden solapar citas por médico y por paciente.
- **Cache y sesión:** el cache falla abierto y consulta PostgreSQL. La sesión falla cerrado: sin Redis nadie entra.
- **Eventos:** el dato y su evento se guardan en la misma transacción. El Worker publica después y el consumidor no duplica notificaciones.
- **Infraestructura:** Docker Compose para QA, Terraform para Azure y Ansible para configurar el servidor.

## 4. Riesgos y prioridades

La matriz completa está en 09-matriz-de-riesgos.md, con 19 riesgos clasificados por confidencialidad, integridad y disponibilidad.

| Prioridad | Qué se prueba primero | Riesgos |
|---|---|---|
| 1 | Doble reserva bajo alta demanda | R-08, R-09 |
| 1 | Acceso a datos clínicos ajenos | R-01, R-02, R-03 |
| 1 | Pérdida de un resultado si cae RabbitMQ | R-12, R-13 |
| 2 | Sesiones, secretos y vulnerabilidades | R-04, R-05, R-06, R-07 |
| 2 | Caída de Redis y saturación | R-14, R-15 |
| 3 | Respaldos y disponibilidad del servidor | R-16, R-17, R-18 |

## 5. Estrategia de pruebas

La mayor parte de las pruebas corre sin interfaz, porque es más rápida y estable. Las pruebas de navegador y móvil cubren los recorridos que usan las personas.

| Nivel | Herramienta | Qué demuestra |
|---|---|---|
| Unitario | xUnit | Reglas de dominio y lógica móvil |
| Integración | xUnit y Testcontainers | API con PostgreSQL, Redis y RabbitMQ reales |
| API | Insomnia (`inso`) | Contrato, errores, autenticación y autorización |
| Web | Cypress | Recorridos por rol en un navegador real |
| Móvil | Maestro | Flujos del paciente en Android |
| SAST | SonarQube, Fortify | Defectos y vulnerabilidades en el código |
| SCA | Dependency-Check, npm audit | Dependencias vulnerables |
| DAST | OWASP ZAP | Ataques sobre el ambiente QA en ejecución |
| Rendimiento | k6 | Umbrales de p95, errores y throughput |
| Accesibilidad | axe-core | Reglas WCAG en 14 pantallas |
| Usabilidad | Sesión con personas y SUS | Facilidad de uso real |

Toda corrección se comprueba repitiendo la prueba que encontró la falla. Los defectos se registran en 12-registro-de-defectos.md.

## 6. Ambientes y datos de prueba

| Ambiente | Uso | Cómo se levanta |
|---|---|---|
| Testcontainers | Integración, con contenedores desechables | `scripts/dotnet-linux.sh test MediCore.slnx` |
| QA local | API, Cypress, ZAP y k6 | `scripts/qa.sh init`, `up`, `seed`, `smoke` |
| CI local | Jenkins (8090) y SonarQube (9000) | `ci/docker-compose.ci.yml` |
| Android | Maestro en emulador | `scripts/maestro_run.py` |
| QA en Azure | Último paso, sujeto a la decisión de pago | `documentacion/06-infraestructura.md` |

Todos los datos son ficticios. `--seed-qa` crea usuarios y datos solo en Development o QA. Las contraseñas de QA viven en `.env.qa`, que no se versiona. Cada corrida de k6 crea 30 médicos, 300 pacientes y 55 fechas para que las agendas no se llenen.

## 7. Pruebas unitarias

Proyectos `tests/MediCore.UnitTests` (dominio) y `tests/MediCore.Mobile.Tests` (lógica de la app).

Cubren reglas de horarios, citas de 30 minutos, estados, validación de recetas y parámetros, renovación de sesión móvil y `Idempotency-Key` en reintentos.

## 8. Pruebas de integración

Proyecto `tests/MediCore.Api.Tests`, con la API real y dependencias reales en contenedores.

| Integración | Pruebas | Qué se verifica |
|---|---|---|
| API con PostgreSQL | `AppointmentsApiTests`, `ClinicalApiTests`, `PatientsApiTests` | Restricciones, transacciones y concurrencia de agenda |
| API con Redis | `AvailabilityApiTests`, `IdentityApiTests` | TTL, datos vencidos, invalidación, sesiones y caída de Redis |
| API con RabbitMQ | `MessagingTests`, `PatientEventsTests` | Outbox y publicación con confirmación del broker |
| Consumidor con RabbitMQ | `MessagingTests` | Reintentos, DLQ, idempotencia y broker detenido |

## 9. Pruebas de API

`tests/insomnia/medicore.insomnia.json` tiene 61 solicitudes y 65 comprobaciones. Se ejecuta con `scripts/api-tests.sh` contra QA.

Cubre casos positivos, negativos, autenticación, autorización por rol y por recurso, y errores. Incluye la API de referencia completa, reservas repetidas, IDOR y los 405 de edición clínica.

## 10. Pruebas funcionales y smoke

- **Smoke:** `scripts/qa.sh smoke` comprueba disponibilidad con PostgreSQL, cabeceras de seguridad, que el proxy no exponga OpenAPI e inicio de sesión de Recepción.

- **Funcionales:** Cypress, en `tests/cypress/e2e`, recorre Recepción, Médico, Laboratorio y Administrador.
- **Negativos:** credenciales inválidas, acceso sin permiso, horario ocupado y formularios incompletos.

Los casos de prueba y su resultado están en 11-casos-de-prueba.md.

## 11. Pruebas de regresión

El pipeline repite todas las suites en cada ejecución. Además:

- Cada defecto de producto corregido deja una prueba que lo reproduce, como el carácter nulo en `PatientsApiTests`.
- Cypress y xUnit se repiten después de cada cambio de código antes de declarar un resultado.

## 12. Pruebas móviles

`tests/maestro` tiene 8 flujos: login válido, login inválido, reserva, consulta y cancelación, resultados, recetas, notificaciones y cierre de sesión. Usa el paciente aislado `e2e.paciente`.

La app usa solo la API real. El APK se compila con `scripts/android-build-linux.sh`.

## 13. Pruebas de seguridad

| Tema del documento | Cómo se prueba |
|---|---|
| RBAC por rol | `IdentityApiTests`, Insomnia y Cypress `acceso-permisos` |
| Autorización por recurso | `ClinicalApiTests`, `LaboratoryApiTests` e Insomnia |
| Datos sensibles | Pruebas de bitácoras, `PatientEventsTests` y revisión de auditoría |
| Sesiones y autenticación | JWT, refresco de un solo uso, bloqueo y límite por IP |
| API Security | Validación, 405, cabeceras, ZAP API con sesión real |
| Trazabilidad | `audit_entries`, `GET /api/audit` y sus pruebas |
| Código y dependencias | SonarQube, Dependency-Check, npm audit y Fortify si hay licencia |

Las excepciones aceptadas viven en `ci/riesgos-aceptados.json`, con su motivo.

## 14. Pruebas de rendimiento

`tests/k6/medicore.js` simula pacientes que reservan mientras profesionales consultan agenda y expedientes. Los umbrales se declararon antes de ejecutar y están en 07-rendimiento.md.

| Perfil | Carga | p95 lectura | p95 reserva | Error inesperado |
|---|---|---|---|---|
| Load | 30 usuarios por 3 min | < 800 ms | < 1000 ms | < 1 % |
| Stress | Hasta 150 usuarios | Informativo | Informativo | < 5 % |
| Spike | De 10 a 200 usuarios | < 2000 ms | < 3000 ms | < 2 % |

Al terminar cada perfil debe haber 0 citas solapadas, 0 eventos pendientes y 0 mensajes en la DLQ.

## 15. Pruebas de usabilidad y accesibilidad

- **Accesibilidad:** `accesibilidad-axe.cy.js` analiza 14 pantallas con axe-core 4.10.3 y reglas WCAG A y AA.
- **Usabilidad:** 08-usabilidad.md define 8 tareas, el cuestionario SUS y los criterios. Exige al menos 5 personas reales.

## 16. Criterios de entrada y salida

**Entrada**

- El código compila y `dotnet format` no reporta cambios.
- Las migraciones se aplican en un ambiente limpio.
- QA responde al smoke y tiene datos sembrados.

**Salida**, comprobada por `scripts/quality-gate.py`

- Quality Gate de SonarQube aprobado.
- Ninguna vulnerabilidad crítica sin aceptar en npm audit o Dependency-Check.
- Ninguna alerta alta sin aceptar en ZAP web o ZAP API.
- Colección de Insomnia sin fallas.
- Umbrales de k6 cumplidos.

Una prueba crítica fallida detiene el pipeline en su etapa, y por eso el despliegue final no se alcanza.

## 17. Gestión de defectos

Cada defecto registra pasos, resultado esperado, resultado obtenido, severidad, prioridad y evidencia.

| Severidad | Significado |
|---|---|
| Crítica | Pérdida o exposición de datos clínicos, o sistema inutilizable |
| Alta | Una función principal falla, sin alternativa |
| Media | Falla con alternativa, o error incorrecto sin daño de datos |
| Baja | Efecto menor o de presentación |

**Prioridad:** P1 corregir antes de la entrega, P2 corregir en esta versión, P3 planificar.

**Ciclo:** abierto, en corrección, verificado y cerrado. El defecto se cierra solo después de repetir la prueba. En GitHub se reporta con `.github/ISSUE_TEMPLATE/defecto.yml`.

## 18. Métricas y resultados

Los resultados de cada herramienta, con su fecha y reporte, están en 13-informe-final.md.

## 19. Riesgos residuales

Se detallan en el informe final y en la matriz de riesgos. Los principales son las copias en el mismo disco (R-16), el servidor único (R-17) y el ambiente de Azure sin aprovisionar (R-18).

## 20. Decisión GO / NO-GO

La decisión y su justificación están en 13-informe-final.md. Se basa solo en los criterios de la sección 16 y en lo que realmente se ejecutó.